SCHOOL OF COMPUTER SCIENCE

September 2025 Semester

Computer Vision and Natural Language Processing

(ITS69204)

Group Assignment (30%)

Due Date: 14 December 2025, 11.59 PM

Submission Channel: myTIMeS

Taylor's University / Taylor's College

ASSESSMENT SUBMISSION DECLARATION FORM

Module Code:	ITS 69204
Module Name:	Computer Vision and Natural Language Processing
Assessment:	Group Assignment
Semester/Year	Sep/2025

Student Name	**Student ID**
Retaj Ahmed [redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

Table of Content

A. Introduction

Due to the increasing prevalence of diet-related chronic diseases, such as obesity, type 2 diabetes, and cardiovascular disorders, the necessity to implement intelligent and real-time dietary monitoring systems has soared. Conventional methods of food logging are based on manual report entry, thus are prone to underreported, subject to estimation error, and low compliance (Ravelli and Schoeller, 2020). Automating food identification by applying computer vision techniques provides a scalable, precise, and user-friendly alternative; forming a basis to integrate with NLP components in the future giving personalized nutritional feedback such as calorie breakdown, glycemic index alerts, or recipe ideas. Recent development verifies that deploying end-to-end AI pipelines is growing, for instance, using fine-tuned CNNs for object recognition and transformer-based NLP for contextual diet reasoning in real-world healthcare and mobile-based nutrition settings (Kirk *et al.*, 2022).

Dataset Description

Dataset Link: Food-11 image dataset

Our dataset title is “Food-11 image”, curated by Tro Lukovich and is publicly available on Kaggle. This dataset is a collection of various food images divided into 11 classes (Bread, Dairy Products, Deserts, Eggs, Fried food, Meat, Noodles-Pasta, Rice, Seafood, Soup, Vegetable-Fruit), including major food categories commonly consumed globally. The dataset is divided into 3 groups: training, validation and evaluation.

Dataset Composition

Table 1 - Dataset Composition Information

Total Classes	**Image Format**	**Color Mode**	**Resolution**	**Images per Class**
11 class	JPEG	RGB	Varies (resized to 224*224)	Limited to 250 image per class for more efficiency and faster training time

Challenges and Limitations

- **Class Imbalance:** Imbalanced distribution reduces model sensitivity to minority classes (e.g., Seafood or Egg classes)
- **Visual Ambiguity:** Overlap between food textures, occlusions (e.g., sauce covering the food), and various types of presentation (e.g., fried vs. boiled eggs); increases the risk of misclassification**.**
- **Limited Diversity:** images are high quality ,most of them taken with smart cameras, which restricts the ability to generalize to real-user scenarios (e.g., blur, glare, irregular plating)

Aim

To develop, implement and evaluate a new CNN food recognition system that is capable of automatically classifying food images into their right categories with a high degree of accuracy, which can be used as a vision of our future real-time CVNLP nutrition assistant

Objectives

- To preprocess, explore and augment the Food-11 dataset to reduce class imbalance and increase robustness.
- Develop CNN architecture optimized for food feature extraction and classification.
- Train and validate the model with relevant regularization and optimization methods.
- Evaluate the model performance using accuracy, precision, recall and confusion matrix.

Motivation and Significance

This project collaborates AI and public health where accurate, low-latency trained food recognition allows to democratize nutrition literacy, particularly within populations with limited access to dietitians (e.g., rural or low-income communities). In terms of academics, it provides practical exposure in developing task-specific CNNs, managing data imbalance, and real-world deployability. The model is pedagogically applicable as a foundation of multimodal systems by which CV defines what is eaten, and NLP defines why and how much is crucial.

Potential Applications

- **Diabetes Management Tools:** applications like Glucose Buddy use real-time food ID to predict postprandial glucose spikes and suggest insulin adjustments. Meals photos can be more accessible and easier for patients to monitor their case.
- **Mobile Fitness Apps:** Implementing the model with apps like MyFitnessPal and Lose it; allows photo-based logging where the user captures a photo of their meal rather than typing the ingredients which enhances system compliance in estimating the calories of meals.
- **Public Health Surveillance:** World Health Organization(WHO) and Food and Agriculture Organization(FAO) can use aggregated, anonymized food photo data to monitor dietary trends among populations.

B. Methodology

Architecture Design

The core architecture design used is transfer learning model, where a pre-trained model (MobileNetV2) is repurposed for a new, specific task (food classification). The model is based on MobileNetV2, a conventional neural network (CNN) initially trained on ImageNet dataset. By leveraging MobileNetV2, the model acquires rich and hierarchical feature representation, simple edges and textures in the initial layer and complex object parts in the deeper layers; applicable for visual recognition tasks (GeeksforGeeks, 2024). Furthermore, adapting this pre-trained model and modifying its final layers is more efficient and effective, as less data and training time are required than creating and training a CNN model from scratch.

Table 2 - MobileNetV2 model (frozen) Architecture Breakdown

Layer Type	**Configuration**	**Justication**
Input Layer	224*224*3	Standard input size for MobileNetV2
		Balances visual detail and computational efficiency
Data Augmentation	RandomFlip RandomRotation(0.1)	Preventing overfitting by increasing dataset variability
Frozen MobileNetV2	Pre-trained weights (ImageNet)	Feature Extraction (without retraining base model)
	include_top = False	
GlobalAveragePooling2D	Pooling over feature mapping	Helps focus on dominant features and adds robustness to shifts.
	Reduces 7×7×1280 to 1280	Reduces dimensions and parameters.

Dense layer 1	units = 1024 activation = relu	Learns complex patterns from the extracted features
Dropout 1	rate = 0.5	Prevents overfitting in dense layer by regularisation
Dense layer 2	units = 512 activation = relu	Further feature refinement before classification
Droupout 2	rate = 0.5	More regularisation to improve generalization
Output Layer	units = 11 activation = soft_max	11 units to match the food-11 image classes
		Softmax outputs interpretable probability scores

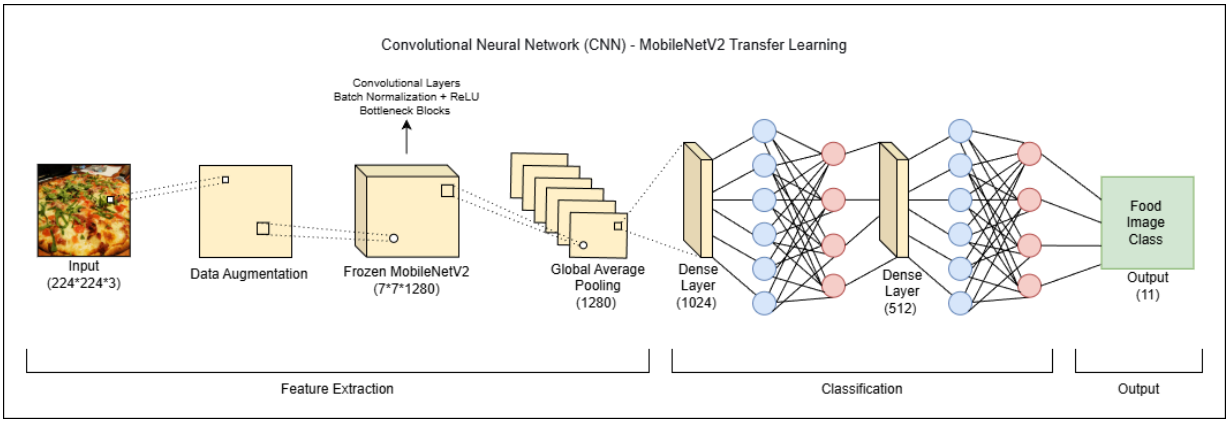

Figure 1 - CNN MobileNetV2 (Frozen) Diagram

Implementation and Code Documentation

- MobileNetV2 (Frozen) - Feature Extraction

```
# Load Pre-trained Base Model (MobileNetV2)
base_model = MobileNetV2(
    weights='imagenet', # Use weights pre-trained on the ImageNet dataset
    include_top=False, # Exclude the final classification layer of MobileNetV2
    input_shape=input_shape
)

# Freeze the Base Model Layers (Feature Extraction Strategy)
base_model.trainable = False
```

Figure 2 - Frozen MobileNetV2 for Feature Extraction

In the initial phase, the MobileNetV2 is used as a fixed feature generator. By removing the original final classification layer using "include_top=False", we are left with the convolutional base that gives the output of a high-dimensional, abstract representation of the input image. By freezing the convolutional base, errors during training will not propagate back through these layers; preventing valuable, general-purpose knowledge from being overwritten or degraded when we start training our new dataset. This reduces the number of trained parameters, making training faster and utilizing memory usage, as model only learn about the new classifiers on the top.

- Custom Head - Classifier

```
# Apply classifier head layers:
# Global Average Pooling reduces the spatial dimensions to 1, preparing for the dense layers
x = GlobalAveragePooling2D(name='global_avg_pool')(x)

# First dense layer with ReLU activation
x = Dense(1024, activation='relu', name='dense_1024')(x)
# Dropout layer for regularization to prevent overfitting
x = Dropout(0.5, name='dropout_1')(x)

# Second dense layer with ReLU activation
x = Dense(512, activation='relu', name='dense_512')(x)
# Dropout layer for regularization
x = Dropout(0.5, name='dropout_2')(x)

# Output layer with 'softmax' activation for multi-class probability distribution
outputs = Dense(num_classes, activation='softmax', name='output_phase1')(x)

model = Model(inputs, outputs, name='mobilenetv2_TransferLearningModel')
```

Figure 3 - Custom head

The new classifier built on top of the frozen feature extractor for our 11-class food classification problem.

➔ Output:

Model Summary

Model: "mobilenetv2_TransferLearningModel"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
data_augmentation (Sequential)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_avg_pool (GlobalAveragePooling2D)	(None, 1280)	0
dense_1024 (Dense)	(None, 1024)	1,311,744
dropout_1 (Dropout)	(None, 1024)	0
dense_512 (Dense)	(None, 512)	524,800
dropout_2 (Dropout)	(None, 512)	0
output_phase1 (Dense)	(None, 11)	5,643

Total params: 4,100,171 (15.64 MB)
Trainable params: 1,842,187 (7.03 MB)
Non-trainable params: 2,257,984 (8.61 MB)
Food Classification Model compiled successfully for 11 classes.

Figure 4 - MobileNetV2 Model Summary Output

- MobileNetV2 (Unfrozen) - Fine-tune

```
# Set Base Model Trainability to TRUE for Fine-Tuning
base_model.trainable = True

# Freeze layers up to the 'start_layer' index
for i, layer in enumerate(base_model.layers):
    if i < (len(base_model.layers) + start_layer):
        layer.trainable = False
    else:
        layer.trainable = True

# Compile the Model for Fine-Tuning
model.compile(
optimizer=Adam(learning_rate=1e-5),
loss='categorical_crossentropy',
metrics=['accuracy']
)
```

Figure 5 - Unfrozen MobileNetV2 for Fine-tunning

After the custom head has been trained on fixed features, we enter the fine-tuning phase. Fine-tune allows for performance refinement, by adapting generic ImageNet features to the specific preciseness of our food dataset, leading to a higher accuracy than using the model at the freezing state as an alone fixed feature extractor. Starting with unfrezzing the model to allow weights to be updated during training using "base_model.trainable = True". As a best practice, we do not

unfreeze the entire base model. Early layers learn general features which are useful for many differnt tasks, deep layers learn more specific info about the dataset. By unfrezzing the deep layers and freezing the early layers, we allow the model to adapt its specialised features to our food dataset; which provides a good balance between adaptability and overfitting risk.

- Model Summary Parameters

```
ft_model.summary(expand_nested=True, show_trainable=True)
```

Figure 6 - Model summary print command for Fine-Tune model

Using model.summary() to generate an overview about the new network architecture. Key highlighted points:

➔ Layer (type): name of each layer in the sequential stack.

➔ Output shape: tensor output shape of each layer, showing how the data is transformed as it moves through the network

➔ Parm: number of trainable weights and bias in each layer

Table 3 - Fine-Tune Model Breakdown for Main Layers

Layer Type	**Configuration**	**Justication**
Input Layer	224*224*3	Same as original input dimension
Data Augmentation	RandomFlip RandomRotation(0.1)	Continue regularisation during fine-tuning
Frozen MobileNetV2	Layers 0-149 (first 150 layer)	Preserving the extraction features levels learned from ImangeNet
Fine Tuned MobileNetV2	Last 4 blocks (layers 150 - 153)	Adapts high level and task specific features for food classes classification to improve performance
	base_model.trainable = True	
GlobalAveragePooling2D	Reduces 7×7×1280 to	Spatial reduction before classification

	1280	
Dense Layer 1	units = 1024 activation = relu	Learns complex patterns from the adapted features
Dropout 1	rate = 0.5	Prevents overfitting in dense layer by regularisation
Dense Layer 2	units = 512 activation = soft_max	Further feature refinement before classification
Dropout 2	rate = 0.5	More regularisation to improve generalization
Output	units = 11 activation = soft_max	

➔ Interpreting Parameters in context of the two phases: Feature Extraction (frozen) and Fine-tune (unfrozen):

```
Total params: 4,100,171 (15.64 MB)
Trainable params: 1,842,187 (7.03 MB)
Non-trainable params: 2,257,984 (8.61 MB)
```

- When base_model = False, the summary is showing a large number of non-trainable parms and a smaller number of trainable parms, confirming that the base model is frozen.

```
Total params: 4,100,171 (15.64 MB)
Trainable params: 2,254,987 (8.60 MB)
Non-trainable params: 1,845,184 (7.04 MB)
```

- When base_model.trainable = True, the summary shows that a large number of parameters have switched from non-trainable to trainable; confirming that the fine-tune is active.

C. Training and Evaluation

Data Loading (Splitting)

```
# Global Setup (Seeds and Parameters)
np.random.seed(42)
tf.random.set_seed(42)

# Image Dimensions and Batching
IMG_HEIGHT = 224
IMG_WIDTH = 224
BATCH_SIZE = 32

# Training Schedule
EPOCHS_PHASE1=15
EPOCHS_PHASE2=10
FINE_TUNE_START_LAYER_INDEX = -4
```

Figure 6 - Data preperation

This code block lays the groundwork of settings and hyper parameters of the entire deep learning project, ensuring the reproducibility and format of the experiment. In order to have the same results each time random operations (e.g. weight initialization, data shuffling etc.) are performed, the random seeds are explicitly defined with both NumPy (np.random.seed(42)) and TensorFlow (tf.random.set_seed(42)). The data input physical size is unified by configuring the height and width of the image to 224 pixels (IMG_HEIGHT = 224, IMG_WIDTH = 224), a required image input size by most of the pre-trained models. Efficiency is maximized in the training process with the BATCH_SIZE of 32 which is the amount of images that are handled at the same time. Lastly, the code stipulates a two-phase training plan: Phase 1 will consist of 15 epochs (EPOCHS_PHASE1) which is probably the training on new layers and Phase 2 will comprise 10 epochs (EPOCHS_PHASE2) of fine-tuning. This fine-tuning is limited to the final four layers of the model as defined by the FINE-TUNE-START-LAYER-INDEX = -4.

```python
# Data Loading
def load_food_dataset(data_path, img_height, img_width, batch_size):
    """
    Handles dataset file extraction and loads image data from
    'training', 'validation', and 'evaluation' subdirectories using
    tf.keras.utils.image_dataset_from_directory.
    """

    if os.path.isfile(data_path) and data_path.endswith('.zip') and zipfile.is_zipfile(data_path):
        extract_path = '/content/food_dataset'  # Temporary extraction directory
        if not os.path.exists(extract_path):
            os.makedirs(extract_path)
        with zipfile.ZipFile(data_path, 'r') as zip_ref:
            zip_ref.extractall(extract_path) # Extract zip contents
        data_path = extract_path

    data_root_dir = os.path.join(data_path)

    # Basic directory checks
    if not os.path.exists(data_root_dir):
      print(f"Training directory not found at {data_root_dir}")
      if os.path.isdir(data_path):
          print("Available directories:", os.listdir(data_path))
      else:
          print(f"{data_path} is not a directory. Please check the path.")

    train_dir = os.path.join(data_root_dir, 'training')
    val_dir = os.path.join(data_root_dir, 'validation')
    test_dir = os.path.join(data_root_dir, 'evaluation')
```

Figure 7 - Data Loading

The given code snippet represents the general design of the load-food-dataset procedure that is essential to ready the food pictures to be sent to the classification model. Its main purpose is to make sure that the data set is found, retrieved, and arranged accordingly. The first step of the code verifies whether data is a compressed zip file and in that case, it decompresses all data into a temporary directory. This is an essential step that will transform the information in one zipped file into a file system format that is easily readable. After ensuring that the data path is a directory, the code identifies the three required subdirectories explicitly namely the train, val and test directories with references to the training partition, the validation partition, and the evaluation partition respectively. This directory configuration is basic, as it enables follow-up processes (such as tf.keras.utils.image_dataset_from_directory, as described in the explanation) to load the images on the disk with ease in batches.

```python
def load_ds(path, subset_name, shuffle):
    """Helper to load one dataset subset (train, val, or test)."""
    print(f"Loading {subset_name} data from {path}")
    ds = tf.keras.utils.image_dataset_from_directory(
        path,
        labels='inferred',
        label_mode='categorical', # Output: one-hot encoded labels
        image_size=(img_height, img_width),
        batch_size=batch_size,
        shuffle=shuffle
    )
    return ds

# Load the three dataset splits
train_ds = load_ds(train_dir, 'Training', shuffle=True)
val_ds = load_ds(val_dir, 'Validation', shuffle=False)
test_ds = load_ds(test_dir, 'Test/Evaluation', shuffle=False)

class_names = train_ds.class_names
NUM_CLASSES = len(class_names)
```

Figure 8 - Load function for subset files

```python
def configure_ds(ds):
    """Applies normalization and performance optimizations (NO CACHING)."""
    # Rescale inputs from [0, 255] to [0, 1] for MobileNetV2 input requirements
    normalization_layer = Rescaling(1./255)
    ds = ds.map(lambda x, y: (normalization_layer(x), y))
    # Prefetching allows the data processing pipeline to work concurrently with model training
    ds = ds.prefetch(buffer_size=tf.data.AUTOTUNE)
    return ds

# Apply configuration to all datasets
train_ds = configure_ds(train_ds)
val_ds = configure_ds(val_ds)
test_ds = configure_ds(test_ds)
```

Figure 9 - Configure function for normalization and performance optimization

In this professional setup, two major performance optimizations can be identified that allow processing large images with thousands of high-resolution images without having to overflow memory (RAM). First, to ensure that large images do not overload memory (RAM), the system is using data streaming using tf.keras.utils.image_dataset_from_directory. The pipeline loads data to the disk batch-by-batch (e.g., 32 images at a time) rather than loading all the data into a single NumPy array which may crash the kernel. Second, the pipeline uses Prefetching (.prefetch(buffer_size=tf.data.AUTOTUNE)) to load all data into the GPU in batches, rather than loading all the data to a single NumPy array, which may crash the kernel. This method makes it possible to process in parallel: as the GPU is processing the current batch (N), the CPU is loading and preparing the next batch (N+1). Such overlap ensures that the maximum possible use of hardware and the idea of the total training time is minimized.

```
Loading Training data from /content/food_dataset/training
Found 9866 files belonging to 11 classes.
Loading Validation data from /content/food_dataset/validation
Found 3430 files belonging to 11 classes.
Loading Test/Evaluation data from /content/food_dataset/evaluation
Found 3347 files belonging to 11 classes.

Data Loading Summary
Found 11 food classes: ['Bread', 'Dairy product', 'Dessert', 'Egg', 'Fried food', 'Meat', 'Noodles-Pasta', 'Rice', 'Seafood', 'Soup', 'Vegetable-Fruit']
Total Training Images Loaded: 9866
Image shape: (224, 224, 3)
```

Figure 10 - Output (successful data loading and preparation)

The generated output ensures that the dataset has been loaded and prepared successfully to the food classification model. A total of 16,643 images (9,866 training images, 3,430 validation images, and 3,347 test images) that were of 11 food classes were dealt with by this process. The 11 classes which reflect the 11 categories that the model is trained to identify are: Bread, Dairy product, Dessert, Egg, Fried food, Meat, Noodles-Pasta, Rice, Seafood, Soup and Vegetable-Fruit. The data will be divided into three important partitions, namely: a large Training set (9,866 images) to learn the model; a smaller Validation set (3,430 images) to be used in miscellany to tune the model; and finally a Test/Evaluation set (3,347 images) to be used to measure the model performance on totally unknown data. Moreover, images are all standardized to (224, 224, 3) where the height and the width have 224 pixels each, and the number 3 is the RGB color channels (a common input size to many pre-trained neural networks such as MobileNetV2).

```python
def display_one_image_per_class(train_dir, class_names, img_height, img_width):
    """
    Reads and plots one sample image from each class directory in the training set
    to visually verify that all classes are present and load correctly.
    """
    print(f"\nDisplaying One Sample Image Per Class ({len(class_names)} Classes)")

    # Determine the best grid size
    num_classes = len(class_names)
    cols = int(np.ceil(np.sqrt(num_classes)))
    rows = int(np.ceil(num_classes / cols))

    plt.figure(figsize=(cols * 3, rows * 3))

    for i, class_name in enumerate(sorted(class_names)):
        class_path = os.path.join(train_dir, class_name)
        if os.path.isdir(class_path):
            file_list = [f for f in os.listdir(class_path) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
            if file_list:
                # Get the first image file found in the directory
                img_path = os.path.join(class_path, file_list[0])

                # Load the image using tf.keras.utils.load_img
                img = tf.keras.utils.load_img(img_path, target_size=(img_height, img_width))
                img_array = tf.keras.utils.img_to_array(img)

                # Plotting
                ax = plt.subplot(rows, cols, i + 1)
                # Rescale the image array from [0, 255] for display
                plt.imshow(img_array.astype("uint8"))

                plt.title(f"{class_name}", fontsize=10)
                plt.axis("off")
```

Figure 11 - Image display function

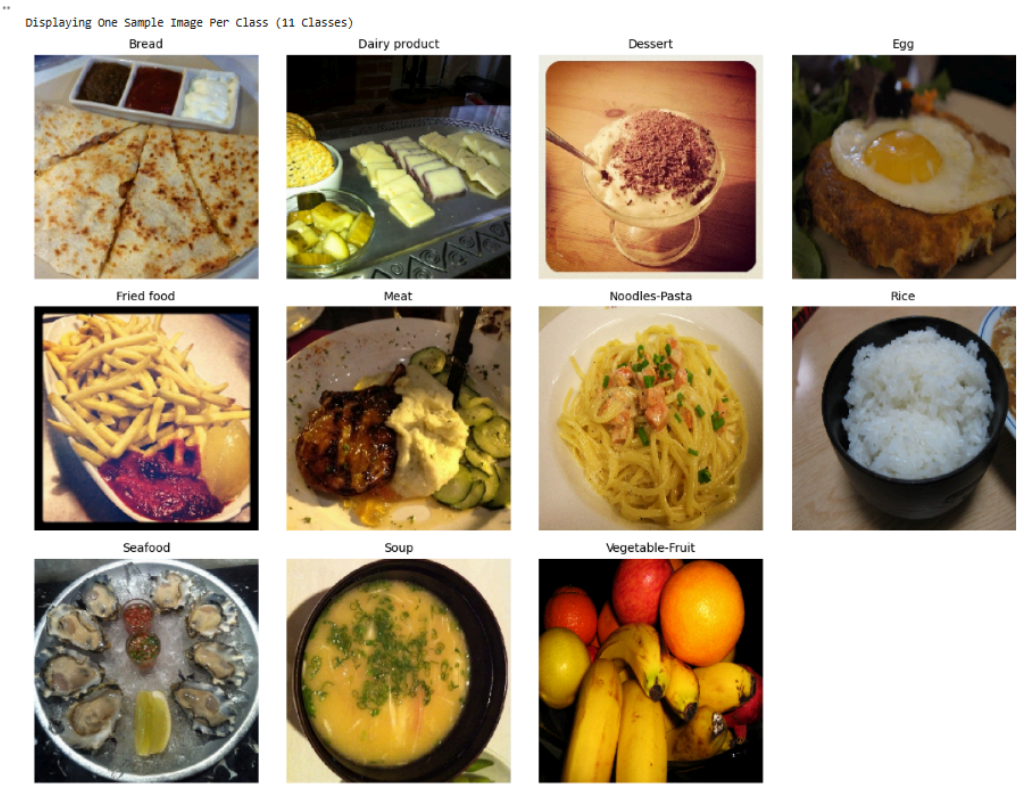

Figure 12 - Output (11 classes image sample)

This is an initial data inspection and verification function. The main purpose of this code is to visually confirm that the data is properly structured and loaded, that every target class is present and contain at least one representative image prior to train a model.

The function iterates through the class_name, which is the subdirectories in the train_dir. It goes through each sub folder, finds the first image file (looking at common image extensions, such as .png, .jpg and jpeg) and loads it by utilizing tf.keras.utils.load_img, resizing it to the predefined img_height and img_width. Then it transforms the image into a NumPy array through tf.keras.utils.img to array. Lastly, the code used matplotlib.pyplot to plot the image into a grid (dimensions being optimized by the use of np.ceil(np.sqrt(num classes))) and name the title to the name of the class.

Training the CNN model

```python
def setup_training_callbacks(patience):
    """
    Configure callbacks for optimal training: EarlyStopping and Learning Rate Reduction.
    EarlyStopping: Stops training when validation accuracy stops improving for 'patience' number of epochs.
    ReduceLROnPlateau: Reduces the learning rate by half if validation loss plateaus (no improvement after 2 epochs).
    """
    print("\nConfiguring Keras Callbacks")

    # Early stopping to prevent overfitting
    early_stopping = EarlyStopping(
        monitor='val_accuracy', # Metric to monitor
        patience= patience, # Number of epochs with no improvement after which training will be stopped
        restore_best_weights=True, # Restores model weights from the epoch with the best monitored value
        verbose=1,
        mode='max' # Look for maximum val_accuracy
    )

    # Learning rate reduction when training plateaus
    lr_reduction = ReduceLROnPlateau(
        monitor='val_loss',
        patience=2, # Number of epochs with no improvement before reducing LR
        factor=0.5, # Factor by which the learning rate will be reduced
        min_lr=1e-6,
        verbose=1,
        mode='min' # Look for minimum val_loss
    )

    return [early_stopping, lr_reduction]
```

Figure 13 - Training callbacks setup

During the Feature Extraction phase, the code starts training of the model with a completely frozen MobileNetV2 base. That is, base_model.trainable = False, which means only the newly added Dense layers in the classifier will receive updates during training. The training loop using model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS_PHASE1, callbacks=callbacks) offers a chance for the model to learn from the Food-11 dataset while monitoring its behavior in terms of validation accuracy and loss through the callbacks defined earlier. The EarlyStopping callback stops the training when the validation accuracy stops improving, which is why the model does not train on all epochs but instead stops at the optimal point. Once training is over, the code extracts and prints final training accuracy and validation accuracy, along with the gap between them. This allows us to determine whether overfitting occurred during Feature Extraction. Since the base model is frozen, training accuracy usually shoots up quickly, while validation accuracy improves more slowly; this behavior is also reflected in the code outputs. This indicates that the classifier might become so well adapted to the dataset that it can practically "guess" the labels, while the frozen layers are stuck in their original state of learning, consequently, Feature Extraction is performance at a level that is just more than the baseline before Fine-Tuning is done.

```python
# Training CNN Model
print(f"\nStarting Training")
print(f"Training for {EPOCHS_PHASE1} epochs.")


callbacks = setup_training_callbacks(patience=3)
history_phase1 = model.fit(train_ds,
                    validation_data=val_ds,
                    epochs=EPOCHS_PHASE1,
                    callbacks=callbacks)

# Analyze training completion
final_epoch = len(history_phase1.history['accuracy'])
final_train_acc = history_phase1.history['accuracy'][-1]
final_val_acc = history_phase1.history['val_accuracy'][-1]
train_val_gap = final_train_acc - final_val_acc

print(f"\nTraining completed at epoch {final_epoch}")
if final_epoch < EPOCHS_PHASE1:
    print("Early stopping activated successfully")

print(f"Final train-validation gap: {train_val_gap:.3f} ({train_val_gap*100:.1f}%)")
```

Figure 14 - CNN model training (feature extraction)

This block of code performs the actual training process for the Feature Extraction model and then evaluates its learning performance. It begins by printing status messages to indicate that training has started and how many epochs the model is scheduled to run. The setup_training_callbacks(patience=3) function is called to attach the EarlyStopping and ReduceLROnPlateau mechanisms, and these callbacks are passed directly into model.fit(), which trains the classifier layers on the training dataset while monitoring performance on the validation set. The history of the training is in history_phase1, from where the code retrieves the final training accuracy, the final validation accuracy, and the total number of epochs completed. Using these, it calculates the difference between the training and validation accuracy, an important metric since this difference can indicate when a model has overfitted in the Feature Extraction phase. The script also checks whether the training stopped early because of EarlyStopping by comparing the actual epoch count to the intended number of epochs. Finally, it prints out the train–validation gap, giving a summary of the model's ability to generalize at the end of Feature Extraction. This metric is the baseline that will later be used to measure the improvements after Fine-Tuning.

The BATCH_SIZE=32 parameter was applied when we used image_dataset_from_directory to load the dataset into the directory, thus the output of the dataset is already batched. Hence no manual batching is required when feeding the image into the model.fit() function.

```
Configuring Keras Callbacks
Epoch 1/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 43s 111ms/step - accuracy: 0.5366 - loss: 1.4784 - val_accuracy: 0.7889 - val_loss: 0.6816 - learning_rate: 0.0010
Epoch 2/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 111ms/step - accuracy: 0.7364 - loss: 0.8063 - val_accuracy: 0.8114 - val_loss: 0.6051 - learning_rate: 0.0010
Epoch 3/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 108ms/step - accuracy: 0.7753 - loss: 0.7110 - val_accuracy: 0.8041 - val_loss: 0.6031 - learning_rate: 0.0010
Epoch 4/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 109ms/step - accuracy: 0.7917 - loss: 0.6567 - val_accuracy: 0.8201 - val_loss: 0.5637 - learning_rate: 0.0010
Epoch 5/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 110ms/step - accuracy: 0.8036 - loss: 0.6111 - val_accuracy: 0.8329 - val_loss: 0.5301 - learning_rate: 0.0010
Epoch 6/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8136 - loss: 0.5752 - val_accuracy: 0.8309 - val_loss: 0.5411 - learning_rate: 0.0010
Epoch 7/15
308/309 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.8113 - loss: 0.5724
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
309/309 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - accuracy: 0.8114 - loss: 0.5724 - val_accuracy: 0.8259 - val_loss: 0.5367 - learning_rate: 0.0010
Epoch 8/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 32s 104ms/step - accuracy: 0.8462 - loss: 0.4678 - val_accuracy: 0.8458 - val_loss: 0.4907 - learning_rate: 5.0000e-04
Epoch 9/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8592 - loss: 0.4279 - val_accuracy: 0.8443 - val_loss: 0.4946 - learning_rate: 5.0000e-04
Epoch 10/15
308/309 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step - accuracy: 0.8637 - loss: 0.4222
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
309/309 ━━━━━━━━━━━━━━━━━━━━ 40s 102ms/step - accuracy: 0.8637 - loss: 0.4222 - val_accuracy: 0.8449 - val_loss: 0.4970 - learning_rate: 5.0000e-04
Epoch 11/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 110ms/step - accuracy: 0.8723 - loss: 0.3751 - val_accuracy: 0.8490 - val_loss: 0.4911 - learning_rate: 2.5000e-04
Epoch 12/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 32s 103ms/step - accuracy: 0.8829 - loss: 0.3530 - val_accuracy: 0.8560 - val_loss: 0.4828 - learning_rate: 2.5000e-04
Epoch 13/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 42s 107ms/step - accuracy: 0.8963 - loss: 0.3252 - val_accuracy: 0.8548 - val_loss: 0.4795 - learning_rate: 2.5000e-04
Epoch 14/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 32s 103ms/step - accuracy: 0.8921 - loss: 0.3186 - val_accuracy: 0.8545 - val_loss: 0.4731 - learning_rate: 2.5000e-04
Epoch 15/15
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8954 - loss: 0.3171 - val_accuracy: 0.8583 - val_loss: 0.4745 - learning_rate: 2.5000e-04
Restoring model weights from the end of the best epoch: 15.

Training completed at epoch 15
```

Figure 15 - Output (CNN model training - Feature Extraction)

This output illustrates the arduous journey of the model through epochs during Feature Extraction training; the training accuracy and validation accuracy gradually climb up, as the model is able to learn better. The performance of the model is summarized for each epoch in terms of accuracy, loss, and learning rate. When the validation loss has not improved for a few epochs, ReduceLROnPlateau is activated, which is indicated by messages like “reducing learning rate to ….” The automatic reduction lowers the learning rate even more so that the model can perform finer updates and not exceed the optimal point when optimizing. The validation accuracy is improving as the training shows that the last validation accuracy is 0.8583, which is the highest over all the epochs. However, the model reaches the predefined epochs, which is 15. In that case, it will halt the training, and the last line, “Restoring model weights from the end of the best epoch: 15,” signifies that the weights were reverted to the epoch with the highest validation accuracy, and it’s also the last-epoch weights in this training. Altogether, this output points to the fact that feature extraction resulted in stable enhancements, with callbacks taking control over the learning rate and stopping the training at the optimal point to avoid overfitting.

```python
# Calculate the starting epoch number for fine-tuning
initial_epoch_ft = history_phase1.epoch[-1] + 1 if history_phase1.epoch else 0

callbacks = setup_training_callbacks(patience=5) # Reconfigure callbacks with patience=5 epochs for Fine-Tuning

print(f"\nStarting Fine-Tuning Training ({EPOCHS_PHASE2} epochs)")
ft_history = ft_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=initial_epoch_ft + EPOCHS_PHASE2, # Total epochs = epochs already run + new fine-tuning epochs
    callbacks=callbacks
)

# Analyze training completion
final_epoch = len(ft_history.history['accuracy'])
final_train_acc = ft_history.history['accuracy'][-1]
final_val_acc = ft_history.history['val_accuracy'][-1]
train_val_gap = final_train_acc - final_val_acc

print(f"\nTraining completed at epoch {final_epoch}")
if final_epoch < (initial_epoch_ft + EPOCHS_PHASE2):
    print("Early stopping activated successfully")

print(f"Final train-validation gap: {train_val_gap:.3f} ({train_val_gap*100:.1f}%)")
```

Figure 16 - CNN model training (fine-tuning)

This code snippet runs the whole fine-tuning process; therefore, it is continuing training from the last phase, hence allowing changes in deeper layers of the model. Here, the initial_epoch_ft variable was assigned the value of the last epoch in history_phase1 plus one, the epoch at which fine-tuning started. To prevent the model from learning everything all at once, it is important to have a progressive training process. The newly introduced callbacks have much longer patience set up for them, which will give the model more time to adjust during this difficult phase.ft_model.fit() then gives the surrender to the fine-tuning process, which trains the model using the training dataset while validating on the validation dataset. The training lasts from initial_epoch_ft to initial_epoch_ft + EPOCHS_PHASE2, thus, the epochs for fine tune training is 15 + 10 = 25 epochs, while all performance metrics-accuracy and loss-are recorded in the ft_history object. At the end of the training, the script pulls out the last train accuracy, last validation accuracy, and total epochs from the stored history. The code also determines whether EarlyStopping has terminated the training before the maximum number of epochs has been reached. When activated, it outputs a confirmation message. Finally, the code calculates and displays the train-validation accuracy gap, indicating how well the fine-tuned model generalizes after the deeper layers were updated. It empowers perfect control over the calibration workflow, directly from the stages of continuing training evaluation in respect of final performance metrics.

```
Starting Fine-Tuning Training (10 epochs)
Epoch 1/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 42s 113ms/step - accuracy: 0.8116 - loss: 0.7564 - val_accuracy: 0.8402 - val_loss: 0.5279 - learning_rate: 1.0000e-05
Epoch 2/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 106ms/step - accuracy: 0.8623 - loss: 0.4734 - val_accuracy: 0.8385 - val_loss: 0.5392 - learning_rate: 1.0000e-05
Epoch 3/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 110ms/step - accuracy: 0.8741 - loss: 0.3799 - val_accuracy: 0.8449 - val_loss: 0.5214 - learning_rate: 1.0000e-05
Epoch 4/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 110ms/step - accuracy: 0.8819 - loss: 0.3744 - val_accuracy: 0.8510 - val_loss: 0.5028 - learning_rate: 1.0000e-05
Epoch 5/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 111ms/step - accuracy: 0.8821 - loss: 0.3633 - val_accuracy: 0.8583 - val_loss: 0.4915 - learning_rate: 1.0000e-05
Epoch 6/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 42s 114ms/step - accuracy: 0.8881 - loss: 0.3347 - val_accuracy: 0.8589 - val_loss: 0.4852 - learning_rate: 1.0000e-05
Epoch 7/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8868 - loss: 0.3473 - val_accuracy: 0.8598 - val_loss: 0.4821 - learning_rate: 1.0000e-05
Epoch 8/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 109ms/step - accuracy: 0.8872 - loss: 0.3274 - val_accuracy: 0.8598 - val_loss: 0.4772 - learning_rate: 1.0000e-05
Epoch 9/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 111ms/step - accuracy: 0.9026 - loss: 0.3075 - val_accuracy: 0.8577 - val_loss: 0.4748 - learning_rate: 1.0000e-05
Epoch 10/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8989 - loss: 0.3048 - val_accuracy: 0.8606 - val_loss: 0.4746 - learning_rate: 1.0000e-05
Epoch 11/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 110ms/step - accuracy: 0.9024 - loss: 0.2939 - val_accuracy: 0.8592 - val_loss: 0.4729 - learning_rate: 1.0000e-05
Epoch 12/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 34s 111ms/step - accuracy: 0.9017 - loss: 0.2964 - val_accuracy: 0.8601 - val_loss: 0.4734 - learning_rate: 1.0000e-05
Epoch 13/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 41s 109ms/step - accuracy: 0.9011 - loss: 0.2944 - val_accuracy: 0.8589 - val_loss: 0.4721 - learning_rate: 1.0000e-05
Epoch 14/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 107ms/step - accuracy: 0.8958 - loss: 0.3034 - val_accuracy: 0.8595 - val_loss: 0.4717 - learning_rate: 1.0000e-05
Epoch 15/25
309/309 ━━━━━━━━━━━━━━━━━━━━ 33s 108ms/step - accuracy: 0.9047 - loss: 0.2899 - val_accuracy: 0.8601 - val_loss: 0.4706 - learning_rate: 1.0000e-05
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 10.

Training completed at epoch 15
Early stopping activated successfully
Final train-validation gap: 0.047 (4.7%)
```

Figure 17 - Output (CNN model training - Fine Tuning)

Unfreezing deeper layers and using minuscule learning rate have been effective ways to showcase the teaming output in fine-tuning. The situation of both training and validation accuracies going up throughout the epochs can be interpreted as the model refining higher-order features instead of learning from the very beginning. However, it is quite normal during fine tuning for the validation accuracy to hover around similar values across several epochs, because the learning rate is kept very small, 1e-5, not to disrupt the pre-trained weights. After the 10th epoch, validation accuracy does not improve for the configured patience triggering EarlyStopping with a message "epoch 15: early stopping.". This is a way to prevent unnecessary training when no further improvements are seen. "Restoring model weights from the end of the best epoch: 10" indicates that the model reverts to the weights of the epoch with the highest validation accuracy and not the latest weights. Finally, the summary reveals that the training was done at epoch 15 and that EarlyStopping was correctly triggered. A difference in test-set performance of 0.047 hardly indicates clear equivalency in the two performance events.

Performance Evaluation and Metrics

In this section, the performance of the model is evaluated using both the Feature Extraction approach and the Fine-Tuning approach. The assessment includes key metrics such as accuracy, precision, recall, F1-score, and the confusion matrix. In addition, the training history is visualized through accuracy and loss plots to better understand the learning behaviour of each model.

1. Defining a Performance Function:

To evaluate the model in a clear and consistent way,a performance function was defined to be used to plot the results of accuracy, loss, and confusion matrices. These functions enable the generation of results to be obtained by merely calling the functions even after every training step, rather than rewriting the same evaluation code. This simplifies the workflow, eliminates repetition, and allows the Fine-Tuned model and the Feature Extraction model to be evaluated through the same evaluation process. The plot history and plot confusion matrix functions will plot the accuracy and loss curves versus epoch, respectively, and assist in visualizing how the model learns with time, and a more detailed description of the prediction errors per class is given by the plot confusion matrix. A combination of these functions form a framework of performance that is organized and reusable to enhance transparency and to be able to compare the two training strategies.

```python
def plot_history(history):
    """Plots training and validation accuracy and loss over epochs."""

    # Extract metrics from the history object dictionary
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']

    # Create the range of epoch numbers for the X-axis
    epochs_range = range(len(acc))

    plt.figure(figsize=(12, 5))

    # Plot 1: Accuracy
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, acc, label='Training Accuracy', color='blue')
    plt.plot(epochs_range, val_acc, label='Validation Accuracy', color='red')
    plt.title('Training and Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend(loc='lower right')

    # Plot 2: Loss
    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, loss, label='Training Loss', color='blue')
    plt.plot(epochs_range, val_loss, label='Validation Loss', color='red')
    plt.title('Training and Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend(loc='upper right')
    plt.show()
```

Figure 18 - Function for Visualizing Training Performance

The first function focuses on visualizing the performance of the model during the training process. It retrieves values of accuracy and loss through the history of training and plots them against the training and validation sets. The function allows one to easily see trends in the accuracy and loss graphs through placing both graphs side by side; this is to see improvements in learning, indications of overfitting, or wavering training behaviour. This concise visual overview can be used to understand that the model is learning or not and that the training environments are correct. The step is best done with a function, which ensures that the process is always similar and the same visualization could be applied to both the Feature Extraction and Fine-Tunning stages.

```python
def plot_confusion_matrix(cm, class_names, title='Confusion Matrix', cmap=plt.cm.Blues):
    """Prints and plots the confusion matrix."""
    plt.figure(figsize=(10, 8))

    # Display the confusion matrix using a color map
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()

    # Set axis ticks and labels
    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names, rotation=45, ha='right')
    plt.yticks(tick_marks, class_names)

    # Normalize the matrix for better visualization
    cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    # Add text annotations to the plot
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], 'd'),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.tight_layout()
    plt.show()
```

Figure 19 - Confusion Matrix Plotting Function

The second function is for generating Confusion Matrix, which gives a detailed account of the model performance performed on each of the individual classes. The table indicates the sum of correct and incorrect predictions in each type of food and it is much easier to pinpoint patterns like which of the classes the model is the most confused with. The visualization presents this data as a heatmap and puts actual values in each cell in order to be clear. This renders the assessment more explicable and identifies certain strong or frail aspects of the classifier. Similar to the above role, the reusability of this step into a reusable function makes sure that both models of the model are tested in the same structured way.

2. Test Set Evaluation

In order to determine the performance of the model on unknown data, the Feature Extraction model and the Fine-Tuning model were tested on the test set. Outputs of the models are classification reports which provide a summary of the precision, recall, F1-score and the overall accuracy of the classification results per category of the food. The results shown in the first figure are the output of the Feature Extraction stage which indicates the performance of the model using exclusively the top layers, when the pre-trained base is frozen. The second snippet shows the result of Fine-Tuning when a portion of the deeper layers of the base model was also modified to enhance the performance. Comparing the two reports, one can note the influence of fine-tuning on individual classes, some groups demonstrate increased accuracy or recall, others do not change. Collectively these evaluation outputs will give a clear picture whether the model generalises well and whether fine-tuning gives significant performance improvements.

Classification Report (Feature Extraction):

	precision	recall	f1-score	support
Bread	0.88	0.79	0.83	368
Dairy product	0.83	0.74	0.78	148
Dessert	0.82	0.79	0.81	500
Egg	0.78	0.86	0.82	335
Fried food	0.81	0.82	0.82	287
Meat	0.89	0.90	0.89	432
Noodles-Pasta	0.99	0.96	0.97	147
Rice	0.90	0.92	0.91	96
Seafood	0.85	0.90	0.87	303
Soup	0.95	0.97	0.96	500
Vegetable-Fruit	0.93	0.96	0.95	231
accuracy			0.87	3347
macro avg	0.88	0.87	0.87	3347
weighted avg	0.87	0.87	0.87	3347

Figure 20 - Classification Report model (Feature Extraction)

The Feature Extraction scheme with a pre-trained model which served as a fixed feature extractor and only a new classification layer was trained resulted in good overall performance with an accuracy of 0.87 and a weighted average F1-score of 0.87. It worked especially when the classes were easily distinguishable with high F1-scores of Noodles-Pasta (0.97), Soup (0.96), and Vegetable-Fruit (0.95). These findings suggest that the general, abstract properties acquired by the base model were also adequate to the correct differentiation of these categories.However, the model showed slightly lower scores in classes with finer-grained discrimination, like Dairy product (F1:

0.78) and Bread (F1: 0.83). This weakness can be explained by the fact that the fixed feature extractor cannot adjust its core layers to the peculiarities of the food image dataset.

Classification Report (Fine-Tuning):

	precision	recall	f1-score	support
Bread	0.82	0.82	0.82	368
Dairy product	0.90	0.75	0.82	148
Dessert	0.86	0.78	0.81	500
Egg	0.81	0.85	0.83	335
Fried food	0.81	0.84	0.82	287
Meat	0.85	0.90	0.87	432
Noodles-Pasta	0.98	0.98	0.98	147
Rice	0.89	0.97	0.93	96
Seafood	0.88	0.90	0.89	303
Soup	0.96	0.96	0.96	500
Vegetable-Fruit	0.93	0.96	0.94	231
accuracy			0.87	3347
macro avg	0.88	0.88	0.88	3347
weighted avg	0.87	0.87	0.87	3347

Figure 21 - Classification Report model (Fine Tuning)

The Fine-Tuning method where the weights of the entire pre-trained model were updated at a reduced learning rate retained the same overall accuracy of 0.87 and a marginally better average F1-score of 0.88. The categories that were initially performing not very good such as Dairy product (F1: 0.82), Seafood (F1: 0.89), and Rice (F1: 0.93) also improved on fine-tuning.It means that the ability to adjust the pre-trained layers to the domain of the particular foods facilitated the model to learn more specific features, which enhanced the result in discrimination between visually similar or under-represented classes. Although the strong classes, such as Soup (F1: 0.96), did not change much, there was a slight drop in other classes, such as Meat (F1: 0.87), which indicates the slight shift of the model focus during the fine-tuning.

Both strategies achieved the same overall accuracy, but Fine-Tuning provided a more balanced and robust classifier. The Feature Extraction model relied on general features, excelling on distinct classes but struggling with subtle categories. Fine-Tuning refined the pre-trained knowledge, resulting in improved F1-scores for weaker classes like Dairy product and Rice, producing a slightly higher macro F1-score (0.88 vs 0.87).

3. Accuracy & Loss Plots

To visualize the learning process and assess how effectively each training strategy optimized the model, the training history is plotted, showing the progression of accuracy and loss over each epoch. The comparison of these plots between the Feature Extraction and Fine-Tuning methods is crucial for understanding the learning dynamics, convergence speed, and the presence of overfitting.

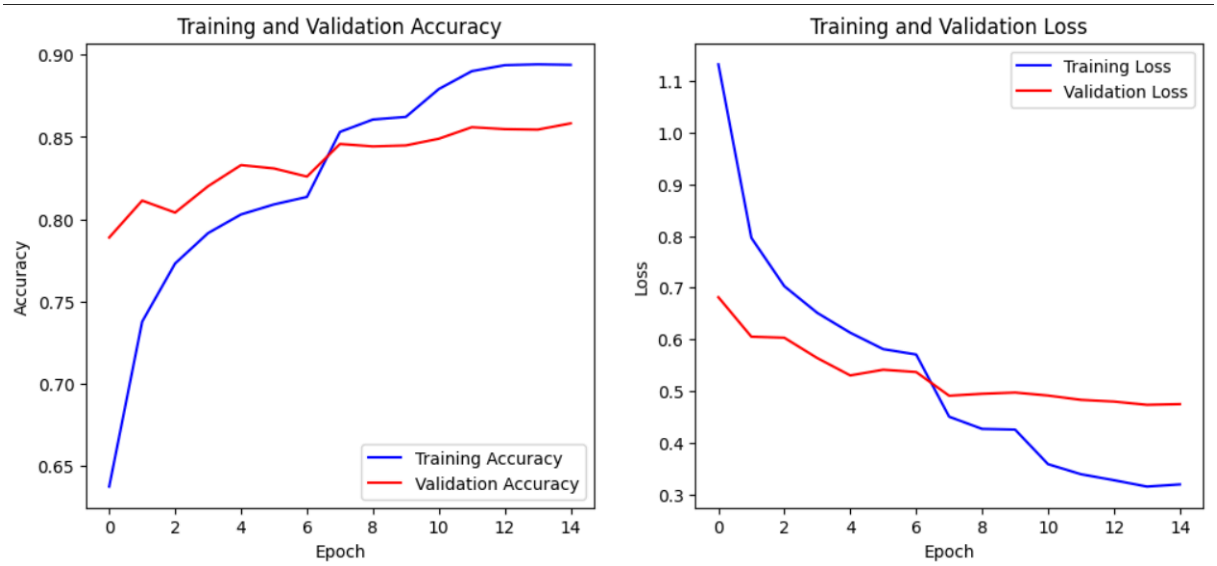

Figure 22 - Training and Validation Accuracy and Loss Plots (Feature Extraction)

The Feature Extraction model demonstrates growing training and validation accuracy at a quick pace at beginning before a gradual rise towards approximately Epoch 8, levelling off at about 0.86 on the validation set. Training Loss goes to 0.3, and the Validation Loss levels to 0.50.The difference between Training Accuracy (= 0.90) and Validation Accuracy (= 0.86) is the start of the overfitting, as is natural, since the base layers are not being updated. The top layers that have just been trained will keep on using the training data to optimize, but not to adapt the deeper features, this limits generalization.

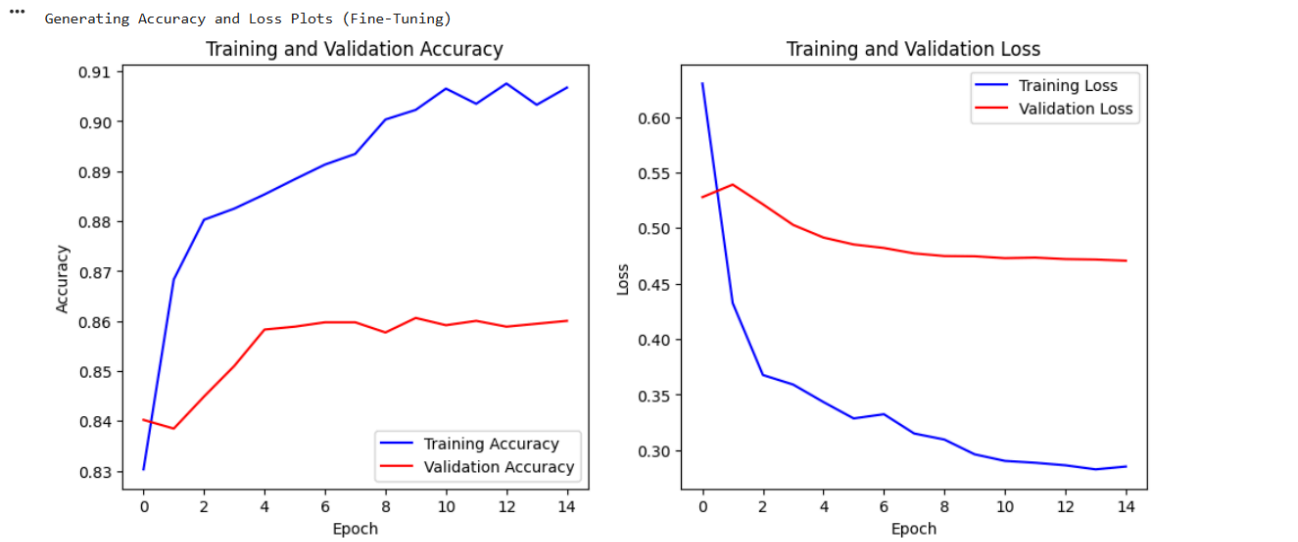

Figure 23 - Training and Validation Accuracy and Loss Plots (Fine tuning)

The Fine-Tuning model has a more controlled and stabilized learning process. Validation Accuracy continuously increases and levels off to 0.86 at Epoch 4, and Training Accuracy steadily increases slowly above 0.90.This is advantageous since the accuracy gain is slower than that in Feature Extraction, but this is also desired: the whole pre-trained network is modified with a small learning rate to fine-tune the basic features. Validation Loss levels off at 0.47 which is a little smaller than Feature Extraction signifying improved generalization.

4. Confusion Matrix

The confusion matrix gives a class-wise analysis of the model performance as it compares the actual labels of the food images against the model projected labels. Contrary to the case of accuracy alone that provides a single value of performance, the confusion matrix points to the specific classes that are well identified and those that are frequently mis-identified. Looking at the diagonal values (correct predictions) and the off-diagonal values (misclassifications), the matrix will illustrate the trends that can be used to describe the strengths and the weaknesses of both training methods.

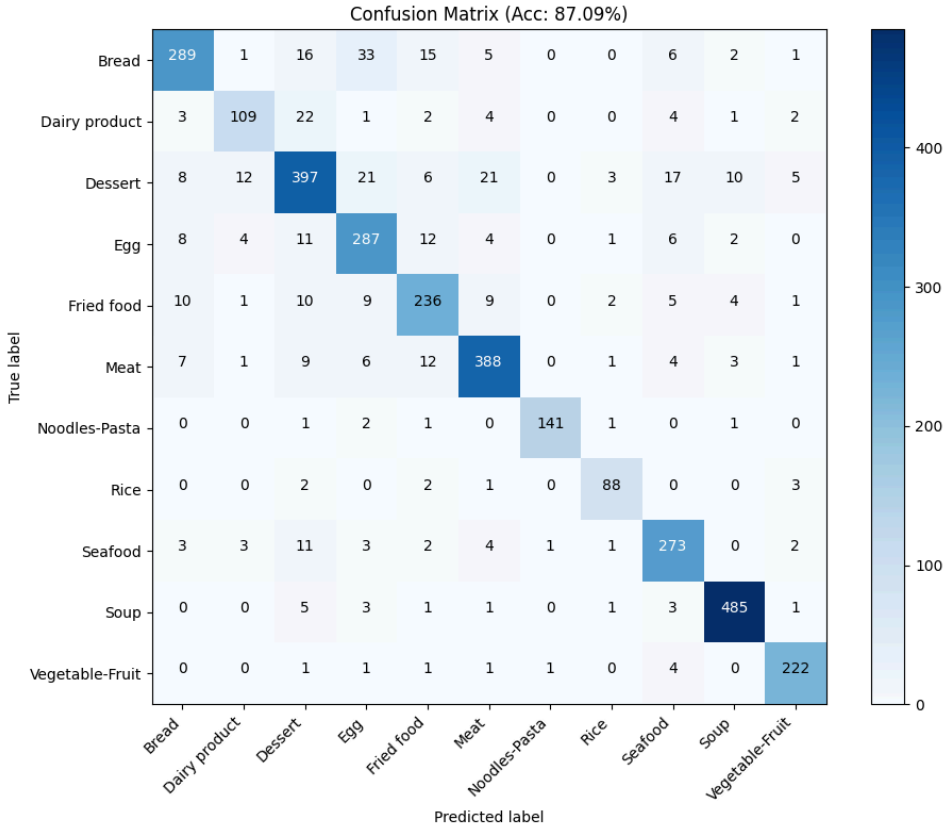

Figure 24 - Confusion Matrix Plot (Feature Extraction)

The confusion matrix for the Feature Extraction model (accuracy: 87.09%) shows that the model effectively identifies several highly distinct classes, including Soup (485 correct predictions), Dessert (397), and Meat (388), demonstrating strong predictive capability. Other classes, such as Rice (88), Noodles-Pasta (141), and Dairy Products (109), have lower counts, reflecting the naturally greater challenge of distinguishing categories with more subtle visual differences. Overall, the matrix highlights the model's ability to capture generalizable features from the pre-trained base while indicating which classes could benefit from additional refinement or fine-tuning.

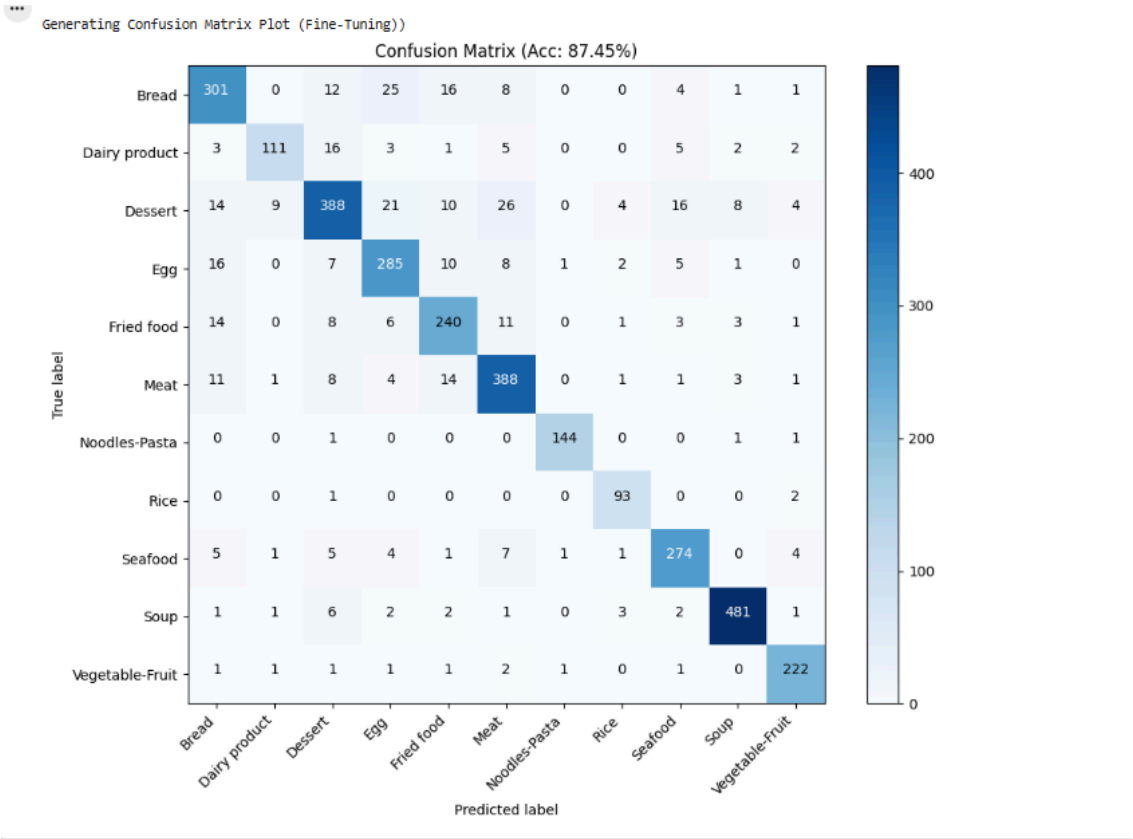

Figure 25 - Confusion Matrix Plot (Fine Tuning)

The confusion matrix for the Fine-Tuning model (accuracy: 87.45%) demonstrates strong performance across multiple classes, with Soup (481 correct predictions), Dessert (388), and Meat (388) achieving the highest counts. Classes such as Rice (93), Noodles-Pasta (144), and Dairy Products (111) have lower counts, reflecting the naturally greater challenge of distinguishing categories with more subtle visual differences. Compared to Feature Extraction, Fine-Tuning shows improvements in some of these lower-count classes, indicating that adapting the pre-trained layers helps the model refine its recognition of visually similar or underrepresented categories. Overall, the matrix highlights a balanced and robust performance across all classes.

The two confusion matrices reveal that Fine-Tuning results in a more balanced and fine-tuned performance in classes. Although the overall accuracy is comparable, the Fine-Tuned model is more precise in most of the classes which were previously more difficult to predict, including Bread and Rice, by enabling deeper layers to adjust to finer visual differences. The Feature Extraction model works well at visually distinct categories, whereas Fine-Tuning increases the capacity of the model to identify classes with less pronounced differentiation, which leads to greater robustness of the model at the class level and generalization.

D. Conclusion

In conclusion, the development,implementation and analysis of a CNN based food recognition system on the food-11 data set, proposed project has been in a position to depict itself as an intelligent real-time dietary monitoring system in response to the rising needs of such systems. The necessity to overcome the limitations of the manual food logging system, such as underreporting and low compliance, to apply the computer vision methods to provide the scalable and accurate food classification, and the opportunities of integrating it with the implementation of the NLP-based food insights, was the initial motivation behind the research. It utilized a transfer learning strategy based on MobileNetV2, which includes a frozen feature extractor and a custom classification head, and, lastly, a fine-tuning phase, which re-trains deep layers to be dataset-specific. It was a design that could facilitate effective training and at the same time accommodated the richness of hierarchical characteristics and circumvented issues such as imbalance among classes, visual ambiguity and lack of image diversity. The training and evaluation phase implied preprocessing of data, increasing data and splitting data on the one hand and systematic performance assessment in terms of accuracy, precision, recall, F1-score, loss, and accuracy plot along with confusion matrices.

The results reveal that not only Feature Extraction but also Fine-Tuning were both highly accurate (up to 87%) .Confusion matrices and learning curves also suggested that the Fine-Tuning model had a superior generalization and tightening down on the hard classes as compared to Feature Extraction which was effective on the easy to discriminate classes.

Overall, the current project not only testifies to the effectiveness of pre-trained CNNs with the incorporation of fine-tuning when applied to food classification but also demonstrates the effectiveness of a highly robust and scalable framework, which can be applied in practice to deliver diabetes management tools, mobile fitness, and dietary monitoring of the population, and a solid foundation upon which future multimodal AI systems can utilize computer vision and natural language processing to offer a comprehensive nutritional assessment and individualized nutrition-related recommendations.

References

- Ravelli, M.N. and Schoeller, D.A. (2020) 'Traditional Self-Reported Dietary Instruments Are Prone to Inaccuracies and New Approaches Are Needed', *Frontiers in Nutrition*, 7(90). Available at: https://doi.org/10.3389/fnut.2020.00090.
- Kirk, D. *et al.* (2022) 'Machine Learning in Nutrition Research', *Advances in Nutrition*, 13(6). Available at: https://doi.org/10.1093/advances/nmac103
- GeeksforGeeks (2024b). *What Is Mobilenet V2?* [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/computer-vision/what-is-mobilenet-v2/.

Marking Rubrics

Section	Criteria	Excellent (90-100%)	Very Good (75-89%)	Good (60-74%)	Fair (40-59%)	Poor (0-39%)	Marks
(a) Introduction (Total: 5 Marks)	**Background & Relevance**	Comprehensive background, well-articulated research goal, and highly relevant.	Clear and relevant background with some depth in research goal.	Background and research goal provided, but lacks depth or some relevance.	Background provided but vague, research goal is unclear or partially relevant.	Poor or no background provided, research goal unclear or irrelevant.	/5
	Objectives	Well-defined objectives, closely aligned with the research goal.	Clear and relevant objectives, mostly aligned with research goal.	Objectives provided but not fully aligned with the research goal.	Objectives are vague or incomplete.	Objectives not mentioned or entirely unclear.	
(b) Methodology (Total: 45 Marks)	**Architecture Design (15 marks)**						

	Design of CNN architecture	Architecture is highly optimized, demonstrating deep understanding of the task requirements and efficient use of layers.	Architecture is well-suited for the task, demonstrating thoughtful consideration of depth, complexity, and optimization.	Architecture is suitable for the task, with adequate depth and complexity, but lacks some optimization.	Architecture demonstrates some understanding of the task but lacks sophistication or optimization.	Architecture lacks depth and complexity, with minimal consideration for the task.	/10
	Justification of design choices	Comprehensive and insightful justification provided, showcasing deep understanding of dataset/task and how design choices address specific challenges.	Detailed justification provided for all design choices, demonstrating clear understanding of dataset/task requirements.	Clear justification provided for most design choices, with some connection to dataset/task characteristics.	Basic justification provided but lacks depth or relevance to the dataset/task.	Minimal or no justification provided for design choices.	/5
	Implementation (30 Marks)						

	Results	Implementation is highly accurate, efficient, and optimized, demonstrating mastery of the chosen deep learning framework.	Implementation is accurate, efficient, and closely follows the proposed architecture.	Correct and mostly complete implementation of the architecture, but with minor issues or inefficiencies.	Basic implementation of the architecture with some inaccuracies or inefficiencies.	Implementation lacks correctness and/or completeness, with significant deviations from the proposed architecture.	/25
	Code documentation	Exceptional comments/documentation provided, offering detailed explanations for all sections, functions, and parameters.	Comprehensive comments/documentation provided throughout the code, making it easy to understand and modify.	Clear comments/documentation provided for most parts of the code, aiding understanding.	Basic comments/documentation provided, but lacks clarity or completeness.	Minimal or no comments/documentation provided, making the code difficult to understand.	/5
C) Training and Evaluation (Total: 30 Marks)	**Dataset Splitting**	Dataset splitting is highly optimized,	Well-justified dataset splitting, leading to balanced and representative	Clear rationale provided for dataset splitting, resulting in reasonably	Basic rationale provided for dataset splitting, but split ratio	Dataset splitting lacks rationale or results in highly imbalanced splits.	/5

		ensuring maximum utilization of data while maintaining balance and representativeness.	training and testing sets.	balanced training and testing sets.	may not be optimal.		
	Training the CNN Model	Training process is highly optimized and thoroughly documented, demonstrating mastery of model training techniques with no errors in code.	Training process is well-documented and includes justification for hyperparameters, showing understanding of model tuning with few minor errors in code.	Training process is clear and mostly complete, with adequate justification for hyperparameters and minimal errors in code snippets.	Training process is demonstrated but lacks some key details or justification for hyperparameters. Some errors might be present in code snippets.	Training process lacks clarity or completeness, with significant errors or omissions in code and hyperparameters not specified.	10/
	Evaluation and Metrics	Evaluation process is highly detailed and thorough,	Evaluation process is well-documented, with clear	Evaluation process is clear and mostly complete, with adequate	Evaluation process is demonstrated but lacks some key	Evaluation process lacks clarity or completeness,	/15

		with comprehensive explanation of evaluation metrics and insightful interpretation of results.	explanation of evaluation metrics and accurate interpretation of results.	explanation of evaluation metrics and some minor errors in code.	details or explanation of evaluation metrics.	with errors in data loading, model prediction, or metric calculation.	
Conclusion (Total: 15 Marks)	**Report Conclusion**	Report is exceptionally well-written and structured, providing comprehensive summaries of approach, challenges, and findings.	Report is well-structured, with detailed summaries of approach, challenges, and findings provided.	Report is clear and well-organized, summarizing approach, challenges, and findings adequately.	Report is somewhat organized but lacks depth in summarizing approach, challenges, and findings.	Report lacks clarity or organization, with minimal detail on approach, challenges, and findings.	/10
	Presentation	Clear, confident delivery by all members; strong visual support; key content	Clear delivery by most members; good visuals; content communicated effectively.	Basic delivery; visuals support content; some sections may lack clarity.	Uneven delivery; weak visuals; some important content missing or unclear.	Poorly delivered or incomplete; visuals missing or ineffective; unclear message.	/10

		well-communicated and engaging.					